KG 구축 how 보완 (2)
Relation 채우기 · 저장 · 검색·답변

따른 문헌의 원리와 실제 수행 과정

2026.11 · 문경민

1. KG 구축 단계와 방법론 구체화 현황

- KG 구축은 온톨로지가 정해 둔 틀(Class, Data Property, Relation)에 자료의 실제 값을 채우는 일이다. 10/30 자료에서는 Class(Entity)와 Data Property(속성값)를 채우는 과정을 정리했다. 그 결과 Entity 264 개와 속성값 616 개가 만들어졌다.
- 이번 자료는 남은 Relation 을 채우는 과정(2 장), 채운 결과를 Neo4j 에 넣는 저장(3 장), 채워진 KG 로 질문에 답하는 검색·답변(4 장)을 같은 방식(따른 원리 → 과정 → 결과)으로 정리했다. 온톨로지와 KG 의 내용은 10/30 자료와 같다.
- 검증은 11/02 회의에 따라 다른 연구들이 KG·온톨로지를 어떻게 검증했는지 조사한 뒤 정리한다. 각 원리의 자세한 설명은 부록 D 에, 10/30 자료까지의 결과는 부록 C 에 있다.

표 1. 단계별 하는 일과 방법론 정리 여부

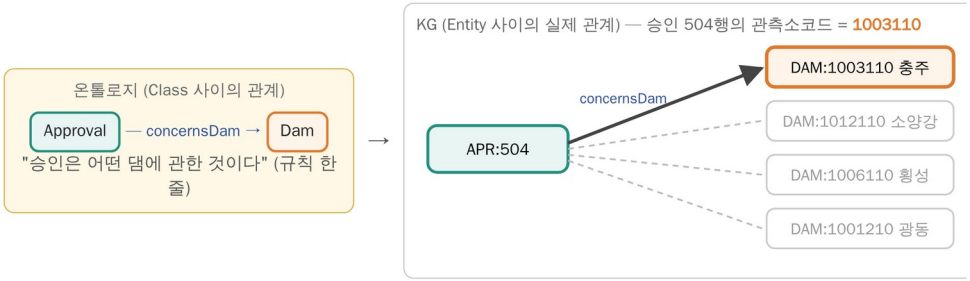
단계	하는 일	방법	방법론 정리
자료 준비	소스 구분과 전처리	—	**정리함 (10/30)**
Class·Data Property 채우기	Entity 와 속성값을 채움	구조화 자료 매핑(정형 표), 비정형 텍스트 추출(비고, 별표 3)	**정리함 (10/30)**
Relation 채우기	Relation 마다 실제 시작·끝 Entity 를 이음	자료 속 키로 끝 Entity 를 정함(댐 표기 맞추기 포함)	**이번 (2 장)**
저장	채운 결과를 Neo4j 에 넣음	CSV → Cypher → Neo4j	**이번 (3 장)**
검색·답변	질문에 맞는 근거를 KG 에서 찾아 LLM 이 답함	의도 추출 → Cypher 템플릿 조회 → 근거 묶음 → 답변	**이번 (4 장)**
검증	만든 KG 가 맞는지 확인	—	**선행연구 조사 후**

2. Relation 채우기

2.1 왜 Relation 을 따로 채우는가

- 그림 1 의 왼쪽은 온톨로지의 Relation 한 줄 "Approval —concernsDam→ Dam"이다. 이것은 Class 사이의 규칙이라 어느 승인이 어느 댐에 관한 것인지는 들어 있지 않다.
- 오른쪽은 KG 이다. 승인 APR:504 에서 댐 4 개로 가는 선은 모두 온톨로지 규칙에 맞는다(점선). 승인 80 개 × 댐 4 개 = 320 가지 조합이 형식상 맞지만, 실제 관계는 80 개뿐이다.

- 실제 관계를 고르는 것은 자료 안의 키이다. 승인 504 행의 관측소코드가 1003110 이므로 APR:504 의 끝 Entity 는 DAM:1003110(충주댐)이다(실선).
- Class 와 Data Property 만 채우면 KG 에는 선이 하나도 없어서 "충주댐의 승인"을 조회해도 아무것도 나오지 않는다. Relation 채우기는 속성 칸에 값을 채우는 것과 같은 일이며, 채우는 값이 숫자나 글자가 아니라 다른 Entity(끝 Entity)라는 점만 다르다.



온톨로지 규칙만 보면 승인 80개 × 댐 4개 = 320가지 조합이 모두 형식상 맞다(점선). 그중 실제 관계 80개를 고르는 근거가 자료 안의 키이다(실선: 관측소코드 1003110 → 충주댐).

그림 1. 온톨로지가 허락한 조합(점선) 가운데 자료 속 키로 실제 관계(실선)를 고른다

Class·Data Property 채우기 다음에 하는 이유

- 끝 Entity 가 먼저 만들어져 있어야 한다. 승인 표를 옮길 때는 다른 자료(K-water 일자료)에서 만드는 댐 Entity 가 아직 없을 수 있다.
- 자료마다 다르게 적힌 댐("충주", 1003110, "충 주 댐")을 같은 댐으로 알아봐야 끝 Entity 를 정할 수 있다(2.3).
- 그래서 모든 Entity 를 만든 뒤에 Relation 을 채운다. 근거 관계(supportedBy)도 온톨로지의 Relation 이므로 이 단계에서 함께 정리한다(10/30 자료 과정 4 의 근거 연결).

2.2 따른 원리와 진행 방식

- 따른 원리: Purohit 외(2024)는 GraphAide 에서 뉴스 기사 1,846 건으로 KG 를 만들 때, 글에서 뽑은 대상의 표기를 기준 지식베이스(DARPA Wikidata)의 식별자(QID)에 대응시켰다. 같은 QID 를 받은 표기는 같은 Entity 가 된다(부록 D.1).
- 원리: 기준 식별자를 하나 정하고, 자료마다 다르게 적힌 같은 대상을 그 식별자에 대응시킨다. 우리는 K-water 댐 코드를 기준 식별자로 삼았다. 대상이 댐 4 개라서 코드 일치와 대응 표로 했고, GraphAide 의 벡터 검색과 LLM 재순위는 쓰지 않았다.
- 참고로 Huang 외(2026)는 댐마다 관련 Entity 를 JSON 파일 하나로 묶어 적재했고, 본문에 서로 다른 자료의 같은 대상을 맞추는 단계는 따로 없다. 우리 자료는 승인 기록, K-water 자료, 규정이 서로 다른 기관에서 따로 와서 댐 표기를 맞추는 과정이 필요하다.
- 진행 방식: 먼저 댐 표기를 기준 식별자에 맞춘다(2.3). 그 다음 Relation 마다 시작 Entity 를 하나씩 읽고, 자료 속 키로 끝 Entity 를 정해 관계 한 줄을 쓰며, 끝 Entity 를 정한 방법(규칙)과 값의 성격을 남긴다. 키가 어느 자료에서 나오는지에 따라 세 가지 방식이 있다(표 2, 2.4~2.6).

표 2. Relation 별 끝 Entity 를 정하는 키

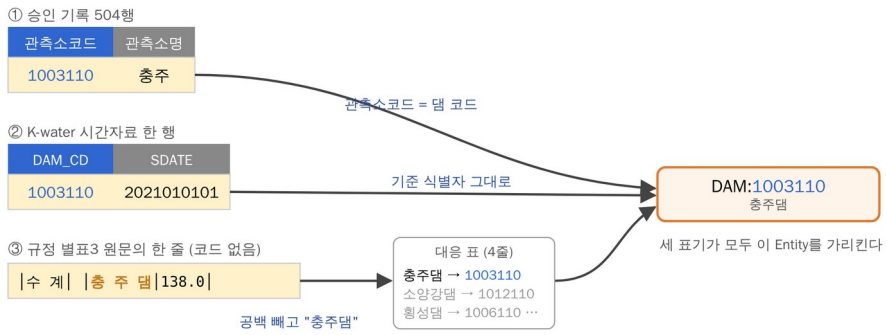
Relation	시작 → 끝	끝 Entity 를 정하는 키	방식	값의 성격	개수
supportedBy	Approval → EvidenceSource	같은 순차번호	2.4 승인 한 행	원본 그대로	80
supportedBy	Operation → EvidenceSource	운영행위 ID 의 순차번호	2.4 승인 한 행	원본에서 유도	68
authorizes	Approval → Operation	같은 순차번호	2.4 승인 한 행	연구용 구성	68
concernsDam	Approval → Dam	관측소코드 = 댐 코드	2.4 승인 한 행	원본 그대로	80
performedOnDam	Operation → Dam	순차번호 → 관측소코드	2.4 승인 한 행	원본에서 유도	68

Relation	시작 → 끝	끝 Entity 를 정하는 키	방식	값의 성격	개수
hasHydrometeorological State	Dam → HMS	같은 댐 코드	2.5 댐 코드	원본 그대로	24
supportedBy	HMS → EvidenceSource	같은 댐 코드	2.5 댐 코드	원본 그대로	24
appliesToDam	Criterion → Dam	별표 3 댐 이름 → 대응표	2.6 별표 3 이름	이름 일치	3
supportedBy	Criterion → EvidenceSource	규정 하나 → 고정 ID	2.6 별표 3 이름	원본 그대로	3
합계					418

값의 성격: 원본 그대로 = 원본의 키가 같아서 이음 / 원본에서 유도 = 원본의 키를 한 번 거쳐 이음 / 연구용 구성 = 원본에 직접 적힌 관계가 아니라 연구에서 정해 이음 / 이름 일치 = 코드가 없어 이름으로 이음.

2.3 댐 표기 맞추기 (개체 해소)

- 그림 2 의 왼쪽은 충주댐이 자료마다 적힌 모습이다. ① 승인 기록은 관측소코드 1003110 과 관측소명 "충주", ② K-water 자료는 DAM_CD 1003110, ③ 규정 별표 3 은 "충 주 댐"이라는 이름뿐이고 코드가 없다.
- ① 은 관측소코드가 K-water 댐 코드와 같아서 그대로 맞춘다(자료 준비 때 4 개 댐 모두 같음을 확인). ② 는 기준 식별자 자체이다.
- ③ 은 비정형 텍스트 추출에서 공백을 빼고 읽은 "충주댐"을 대응 표(표준 댐명 → 댐 코드, 4 줄)에서 찾아 1003110 을 얻는다. 광동댐은 별표 3 에 없다.
- 결과: 세 표기가 모두 하나의 Entity DAM:1003110 을 가리킨다. 이후 2.4~2.6 에서 댐을 끝 Entity 로 정할 때 이 대응을 쓴다.



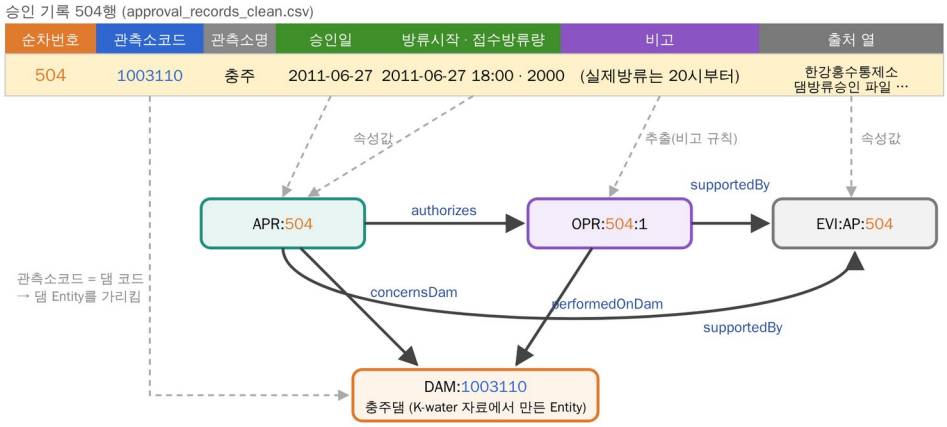
원리 · Purohit 외(2024) — 기준 식별자(QID)를 정하고, 자료마다 다르게 적힌 같은 대상을 그 식별자에 대응시킨다. 우리의 기준 식별자는 K-water 댐 코드이다.

그림 2. 자료마다 다른 충주댐 표기 세 가지를 기준 식별자 DAM:1003110 에 맞춘다

2.4 승인 기록 한 행으로 채우는 관계

- 표 2 의 9 줄 중 5 줄은 승인 기록의 한 행에서 끝 Entity 가 정해진다. 그림 3 의 위쪽은 승인 504 행을 칸 그대로 그린 것이다.
- 점선은 칸에서 Entity 로 가는 길이다(10/30 단계). 승인일·방류시작·접수방류량 칸으로 APR:504 를, 비고를 추출해 OPR:504:1 을, 출처 열로 EVI:AP:504 를 만들었다. 세 ID 에 모두 이 행의 순차번호 504(주황)가 들어 있다.
- 실선은 이번에 채우는 관계이다. ① APR:504 → OPR:504:1(authorizes), ② APR:504 → EVI:AP:504 와 OPR:504:1 → EVI:AP:504(supportedBy)는 ID 의 순차번호가 같은 것끼리 잇는다. ③ APR:504 → DAM:1003110(concernsDam)과 OPR:504:1 → DAM:1003110(performedOnDam)은 이 행의 관측소코드 1003110(파랑)으로 끝 Entity 를 정한다.

- authorizes 는 "같은 승인 행의 비고에서 뽑은 행위를 그 승인이 허가했다"고 연구에서 정해 이은 것이라 값의 성격이 연구용 구성이다(검토할 점 1).
- 결과: 승인 80 행에 같은 방법을 적용해 supportedBy 148 개(승인 80, 운영행위 68), authorizes 68 개, concernsDam 80 개, performedOnDam 68 개를 채운다. 변경 승인 17 개는 운영행위가 없어 authorizes 가 없다.

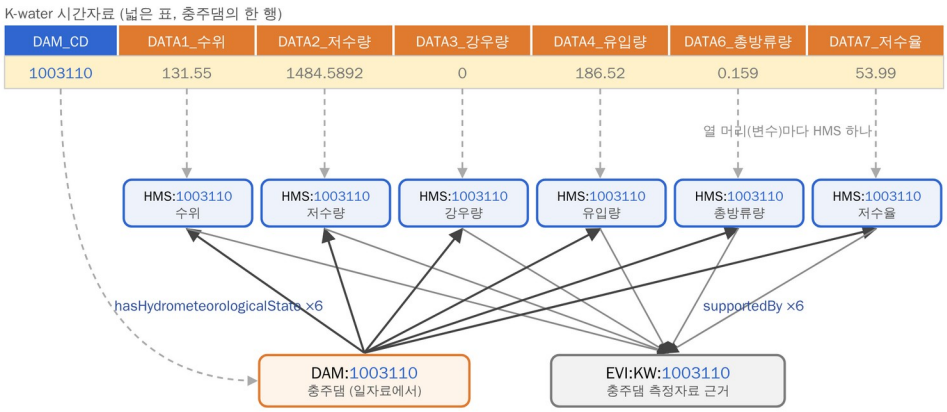


한 행에서 Entity 3개(APR, OPR, EVI:AP)가 나오고, 세 ID에 모두 순차번호 504가 들어간다. 그래서 순차번호만 보면 같은 행에서 나온 Entity끼리의 관계(authorizes, supportedBy)를 바로 정할 수 있다. 댐은 그 행의 관측소코드 1003110으로 정한다(concernsDam, performedOnDam).

그림 3. 승인 504 행 하나에서 채우는 관계 — 순차번호 504 와 관측소코드 1003110 이 끝 Entity 를 정한다

2.5 댐 코드로 채우는 관계

- 그림 4 의 위쪽은 K-water 시간자료의 충주댐 한 행이다. 10/30 단계에서 이 표의 DAM_CD 와 열 머리(변수)로 HMS 6 개를, 댐마다 측정자료 근거 EVI:KW 하나를 만들었고, 댐 Entity 는 일자료에서 만들었다(점선).
- 세 종류 Entity 의 ID 에 모두 같은 댐 코드 1003110(파랑)이 들어 있다. 그래서 ① DAM:1003110 → HMS 6 개 (hasHydrometeorologicalState), ② HMS 6 개 → EVI:KW:1003110(supportedBy)을 코드가 같은 것끼리 잇는다. 다른 자료를 찾을 필요가 없다.
- 결과: 4 개 댐 × 6 개 변수로 hasHydrometeorologicalState 24 개, supportedBy 24 개를 채운다.



세 종류 Entity의 ID에 모두 같은 댐 코드 1003110이 들어 있어, 코드가 같은 것끼리 잇는다.

그림 4. 댐 코드로 채우는 관계 — 같은 코드 1003110 을 가진 Dam, HMS, 측정자료 근거를 잇는다

2.6 별표 3 댐 이름으로 채우는 관계

- 그림 5 의 위쪽은 비정형 텍스트 추출로 만든 중간 파일 criterion_list.csv 의 한 줄이다. 10/30 단계에서 이 줄로 CRI:CR-02 를 만들었다(점선).
- 별표 3 에는 댐 코드가 없다. 그래서 ① 이 줄의 댐 이름 "충주댐"을 대응 표에서 찾아 댐 코드 1003110 을 얻고, ② CRI:CR-02 → DAM:1003110(appliesToDam)을 쓴다. 코드가 아니라 이름으로 정한 끝 Entity 이므로 값의 성격을 "이름 일치"로 남긴다.

- ③ 근거는 규정 하나뿐이라 근거 Entity 도 하나(고정 ID EVI:LAW:별표 3)이다. 기준 3 개 모두 CRI → EVI:LAW:별표 3(supportedBy)으로 잇는다.
- 결과: appliesToDam 3 개(충주, 소양강, 횡성), supportedBy 3 개를 채운다.

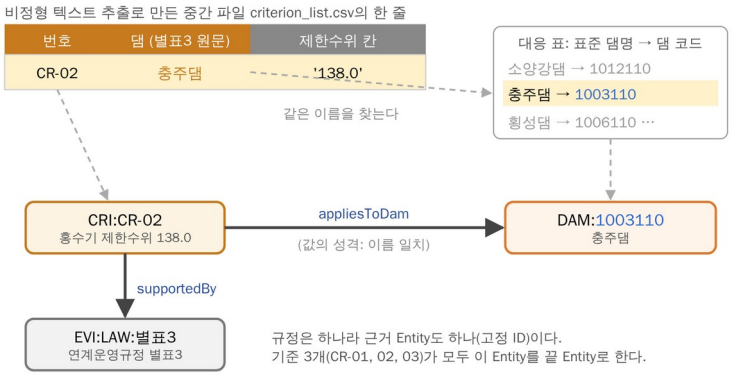


그림 5. 별표 3 댐 이름으로 채우는 관계 — 이름을 대응 표로 댐 코드에 맞춘다

2.7 결과: Relation 이 채워진 CSV

- Relation 9 종(온톨로지의 관계 9 개)의 실제 관계 418 개가 kgv4_relations.csv 에 한 줄씩 채워진다. 그림 6 은 그 파일의 실제 줄이다.
- 한 줄은 Relation 하나이며, 시작 Entity(subject), Relation 이름(relation), 끝 Entity(object), 끝 Entity 를 정한 규칙(rule), 값의 성격(origin), 설명(note)으로 이루어진다. 예를 들어 "APR:504, concernsDam, DAM:1003110"은 승인 504 가 충주댐에 관한 것이라는 관계 하나이다.
- 이로써 KG 에 들어갈 내용은 CSV 파일 세 개에 모두 채워졌다. kgv4_entities(Entity 와 Class), kgv4_properties(Data Property 값), kgv4_relations(Relation)이다. 이 세 파일을 그래프로 저장하는 것이 3 장이다.
- 연결 확인(10/19 질문 "연결이 실제로 되나"): 4 개 댐 전체에서 승인 80 개 모두 댐과 근거에, 운영행위 68 개 모두 댐·승인·근거에, HMS 24 개 모두 댐과 근거에, 기준 3 개 모두 댐과 근거에 이어졌다. 승인 중 운영행위로 이어진 것은 63 개이며, 나머지 17 개는 변경 승인이다.

kgv4_relations.csv — 전체 418줄 중 충주댐의 Entity 몇 개(승인 504, 운영행위 504:1, 수위 HMS, 기준 CR-02, 댐)가 시작인 줄 14개

subject (시작 Entity)	relation	object (끝 Entity)	rule	origin (값의 성격)	note
DAM:1003110	hasHydrometeorologicalState	HMS:1003110:수위	MR-HMS	real	
HMS:1003110:수위	supportedBy	EVI:KW:1003110	MR-SUP-HMS	real	
DAM:1003110	hasHydrometeorologicalState	HMS:1003110:저수량	MR-HMS	real	
DAM:1003110	hasHydrometeorologicalState	HMS:1003110:강우량	MR-HMS	real	
DAM:1003110	hasHydrometeorologicalState	HMS:1003110:유입량	MR-HMS	real	
DAM:1003110	hasHydrometeorologicalState	HMS:1003110:총방류량	MR-HMS	real	
DAM:1003110	hasHydrometeorologicalState	HMS:1003110:저수율	MR-HMS	real	
APR:504	supportedBy	EVI:AP:504	MR-SUP-APR	real	
APR:504	concernsDam	DAM:1003110	MR-APR-DAM	real	승인 레코드의 관측소코드 = 댐 코드 (DAM_CD)
OPR:504:1	performedOnDam	DAM:1003110	MR-OPR-DAM	derived	승인 행의 시설 코드
APR:504	authorizes	OPR:504:1	MR-AUTH	constructed	같은 승인 행의 비고에서 분류한 행위 (연구용 구성, 실행 확인 아님)
OPR:504:1	supportedBy	EVI:AP:504	MR-SUP-OPR	derived	
CRI:CR-02	appliesToDam	DAM:1003110	MR-CRI-DAM	name-match	별표3 시설명 일치(코드 없음)
CRI:CR-02	supportedBy	EVI:LAW:별표3	MR-SUP-CRI	real	
… 나머지 404줄					

그림 6. kgv4_relations.csv 의 실제 줄 — 충주댐 Entity 몇 개가 시작인 줄 14 개

3. 저장

3.1 따른 원리와 진행 방식

- 따른 원리: Huang 외(2026)는 추출한 지식을 JSON 으로 모은 뒤, 온톨로지의 관계에 따라 Python 스크립트가 JSON 의 키-값을 읽어 Cypher 문을 만들고, Entity 와 Relation 을 Neo4j 에 한꺼번에 넣었다. Neo4j 는 Cypher 로 조회·추가·삭제·수정이 효율적이고 여러 Relation 을 건너는 질의에 강해서 골랐다고 밝혔다(부록 D.2).
- Neo4j 는 Entity 와 Relation 을 그래프(점과 선) 모양으로 저장하는 데이터베이스이다. Cypher 는 Neo4j 에 Entity 와 Relation 을 넣고 찾을 때 쓰는 명령 문장이다. CSV 는 줄의 목록이라 "충주댐에서 출발해 관계를 따라간다"는 질의를 바로 할 수 없어서, 그래프로 저장한다.
- 우리는 모은 형태가 JSON 이 아니라 2 장까지 채운 CSV 파일 세 개이다. Python 스크립트가 이 파일의 줄을 읽어 Cypher 문을 쓰고(과정 1·2), 그 문을 모은 파일을 Neo4j 에서 실행한다(과정 3). 출처 파일(kgv4_provenance)은 Neo4j 에 넣지 않고 CSV 로 둔다.
- 아래 그림들은 위에 CSV 의 줄, 가운데에 Cypher 문의 틀, 아래에 실제 Cypher 문과 Neo4j 에 생긴 결과를 둔다. 틀의 { } 자리에는 그 이름의 CSV 칸 값이 들어가며, 화살표는 칸에서 그 자리로 간다.

3.2 CSV → Cypher → Neo4j

과정 1. Entity 를 Cypher 문으로 쓰고 Neo4j 에 넣기

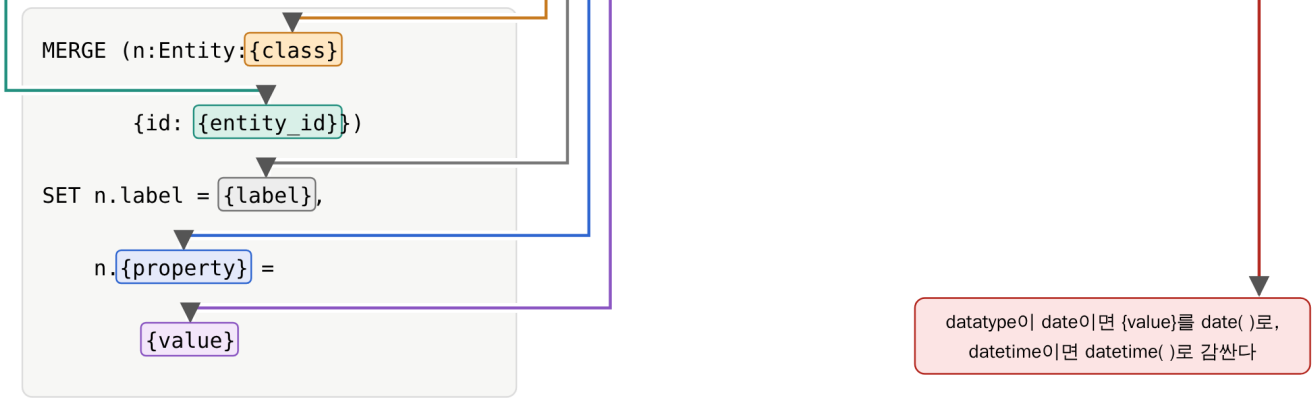
- 원리: Entity 하나는 Cypher 문 하나가 되고, 실행하면 Neo4j 에 Entity 하나가 생긴다. Class 와 Data Property 값도 그 문 안에 함께 들어간다.
- 그림 7 의 맨 위는 입력이다. 왼쪽은 kgv4_entities.csv 의 APR:504 줄, 오른쪽은 kgv4_properties.csv 의 APR:504 줄 2 개이며, Python 스크립트가 이 줄들을 Entity ID 로 모은다.
- 가운데는 Cypher 문의 틀이다. 화살표를 따라가면 entity_id 칸은 {entity_id} 자리에, class 칸은 :Entity 뒤의 {class} 자리에, label 칸은 n.label = 뒤에 들어간다. properties 의 줄마다 property 칸은 n. 뒤의 {property}에, value 칸은 = 뒤의 {value}에 들어가 한 줄씩 생긴다. datatype 칸이 date 이면 값을 date( )로 감싸 글자가 아니라 날짜로 넣는다. 그래야 4 장에서 "2020 년 8 월 안"처럼 기간으로 비교해 찾을 수 있다.
- 그 아래가 APR:504 의 값을 넣어 만든 실제 Cypher 문이고, 칠한 부분이 CSV 에서 온 값이다. MERGE 는 이 Entity 가 없으면 만들고 있으면 그대로 쓰라는 뜻이라서, 같은 문을 다시 실행해도 APR:504 가 둘이 되지 않는다. SET 은 Data Property 값을 넣으라는 뜻이다.
- 맨 아래가 실행 결과이다. Neo4j 에 Entity APR:504 가 Class 와 Data Property 값을 가진 채 생긴다. 264 개 Entity 마다 이런 문이 하나씩 쓰인다.

kgv4_entities.csv (APR:504 줄)

entity_id	class	label
APR:504	Approval	승인 504 (충주댐)

kgv4_properties.csv (APR:504 줄 2개)

entity_id	property	value	datatype
APR:504	approvalTime	2011-06-27	date
APR:504	approvalContent	방류시작 2011-06-27 18:00; 접수방류량 2000㎥/s; 비고: (실제방류는 20시부터)	string



{ } 자리에 CSV 칸의 값이 들어간다. 맨 아래 두 줄(n.{property} = {value})은 properties의 줄마다 생긴다.

↓ APR:504의 줄 3개를 넣으면 (실제 파일의 한 줄)

```
MERGE (n:Entity:Approval {id: 'APR:504'})
SET n.label = '승인 504 (충주댐)',
    n.approvalTime = date('2011-06-27'),
    n.approvalContent = '방류시작 2011-06-27 18:00; …';
```

MERGE — 이 Entity가 없으면 만들고 있으면 그대로 쓴다. ( ) 하나가 Entity 하나이고 n은 이 문 안에서 부르는 이름이다.
:Entity — 모든 Entity에 붙는 공통 표시로, Class를 몰라도 ID 하나로 찾게 한다. SET — Data Property 값을 넣는다.

↓ Neo4j에서 실행하면

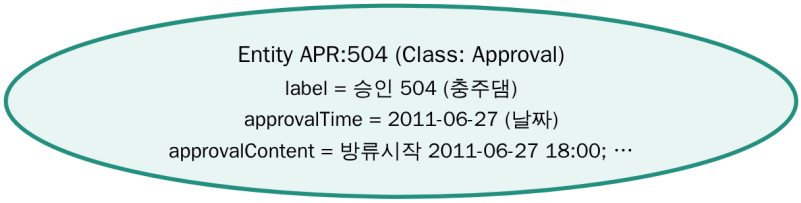


그림 7. 과정 1 — CSV 칸이 Cypher 문의 자리로 들어가고, 실행하면 Neo4j 에 Entity 가 생긴다 (APR:504)

과정 2. Relation 을 Cypher 문으로 쓰고 Neo4j 에 넣기

- 원리: Relation 하나는 Cypher 문 하나가 된다. Relation 은 두 Entity 사이에 긋는 것이므로, 문은 두 Entity 를 먼저 찾고 그 사이에 Relation 을 만든다.
- 그림 8 의 맨 위는 kgv4_relations.csv 의 한 줄이다. 가운데 틀에서 화살표를 따라가면 subject 와 object 칸은 MATCH 문에서 찾을 두 Entity 의 ID 자리({subject}, {object})에, relation 칸은 화살표 모양 안의 {relation}에, rule, origin, note 칸은 SET 뒤의 값 자리에 들어간다. 그 아래가 이 줄의 값을 넣은 실제 Cypher 문이다.

- MATCH 는 이미 있는 Entity 를 찾으라는 뜻이고, 찾은 두 Entity 를 이 문 안에서 a 와 b 로 부른다. (a)-[r:concernsDam]->(b)는 a 에서 b 로 가는 concernsDam Relation 이라는 뜻이며, MERGE 로 없을 때만 만든다.
- 맨 아래가 실행 결과이다. 두 Entity 사이에 방향이 있는 선 하나가 생기고, 그 선에 규칙과 값의 성격이 남는다. 418 개 Relation 마다 이런 문이 하나씩 쓰인다.

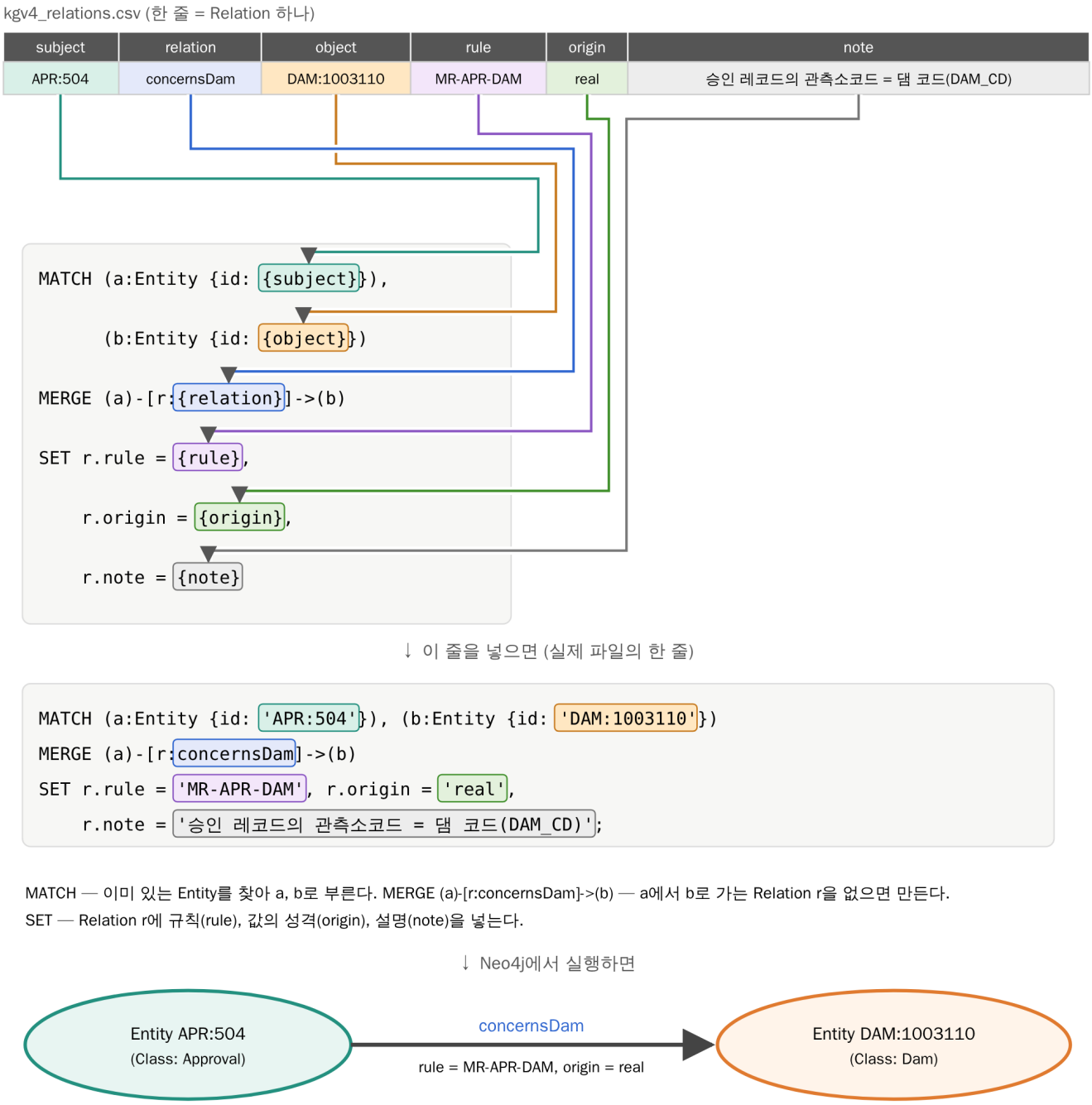


그림 8. 과정 2 — 관계 CSV 한 줄이 Cypher 문이 되고, 실행하면 두 Entity 사이에 Relation 이 생긴다

과정 3. Cypher 문을 모아 Neo4j 에서 실행하기

- 원리: 과정 1·2 에서 쓴 문을 파일 하나에 모아, Neo4j 가 위에서부터 한 번에 실행하게 한다.
- 그림 9 의 왼쪽이 그 파일(kgv4_load.cypher, 684 줄)이다. 주석 1 줄, ID 가 겹치지 않게 막는 규칙 1 줄, Entity 문 264 줄, Relation 문 418 줄 순서이다. Relation 문은 두 Entity 를 찾아야 하므로 Entity 문이 모두 끝난 뒤에 온다.
- 오른쪽은 위에서 아래로 실행 순서이다. Neo4j 를 켜고, 이전에 넣은 Entity 와 Relation 을 모두 지운 뒤, 파일을 실행하고, 마지막에 Entity 와 Relation 수를 세어 264 개, 418 개인지 확인한다.
- 매번 지우고 처음부터 넣으므로, CSV 를 고친 뒤 다시 실행하면 Neo4j 의 KG 가 CSV 와 항상 같아진다.

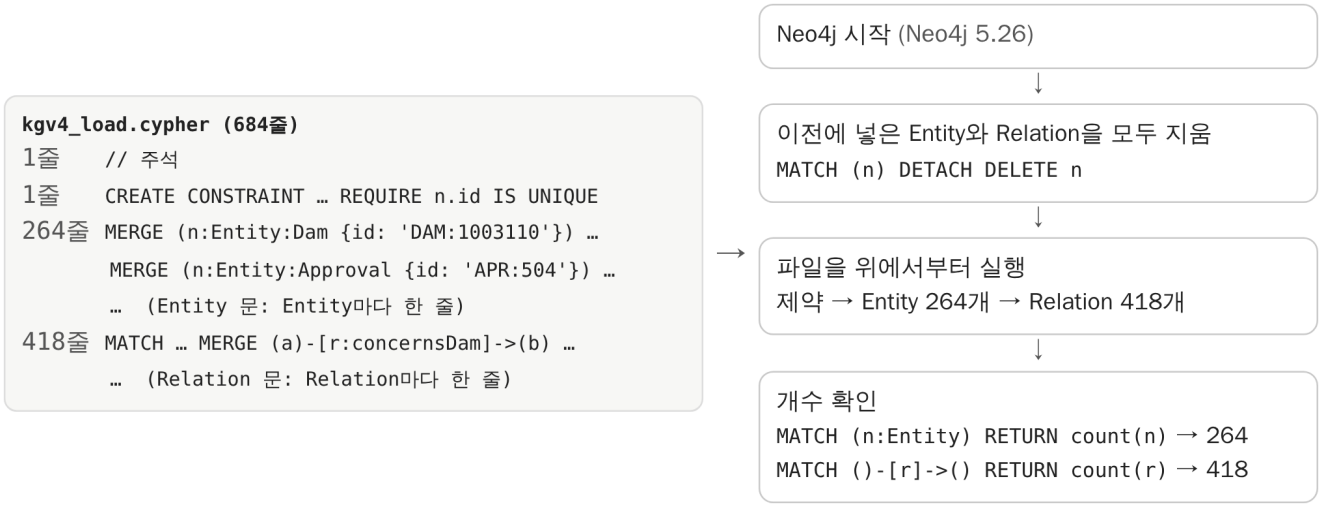


그림 9. 과정 3 — Cypher 파일의 구성과 실행 순서

3.3 KG 밖에 두는 것: 측정 저장소와 문서 저장소

- 시간별 측정값(508,246 행)과 규정 원문은 KG 에 넣지 않고 CSV 파일로 둔다. KG 에는 그 파일의 행을 찾아갈 값만 Data Property 로 둔다.
- 그림 10 의 위는 측정 저장소이다. Entity HMS:1003110:수위의 stationCode(1003110)가 DAM_CD 열과 같고, 찾을 시각(2020-08-02 18:00)이 SDATE_raw 열(2020080218)과 같은 행을 찾는다. 그 행의 DATA1_수위 열이 그 시각의 수위 139.55 EL.m 이다.
- 아래는 문서 저장소이다. Entity EVI:LAW:별표 3 의 sourceLocator("별표 3")가 sourceLocator 열과 같은 행을 찾으면, 그 행의 text 가 규정 원문이다.

측정 저장소 — KG의 Entity가 가진 값(stationCode, 시각)으로 CSV의 행을 찾는다

Entity HMS:1003110:수위
stationCode = 1003110
variableType = 수위

→ DAM_CD = stationCode, SDATE_raw = 찾는 시각(2020-08-02 18:00 → 2020080218), 열 = variableType이 든 열(DATA1_수위)

kwater_mywater_hydr_H_aligned_wide.csv.gz (2010-08~2020-12) · kwater_mywater_hydr_H_wide.csv.gz (2021-01~2026-09)도 같은 열 · 4개 댐 합계 508,246행

source_file	retrieval_time_utc	param1_resolution	DAM_CD	SDATE_raw	DATA1_수위(EL.m)	DATA2_저수량(MCM)	DATA3_강우량(mm)	DATA4_유입량(CMS)	DATA5_source_label_unknown	DATA6_총방류량(CMS)	DATA7_저수율(%)
01_raw/KWater/mywater_aligned/hydr/H/1003110/2020-07-09_2020-08-07.json	2026-10-09T01:36:21Z	H	1003110	2020080217	139.37	2111.215	0.711	4693.654	4693.654	707.265	76.8
01_raw/KWater/mywater_aligned/hydr/H/1003110/2020-07-09_2020-08-07.json	2026-10-09T01:36:21Z	H	1003110	2020080218	139.55	2126.472	1.901	4945.201	4945.201	707.145	77.3
01_raw/KWater/mywater_aligned/hydr/H/1003110/2020-07-09_2020-08-07.json	2026-10-09T01:36:21Z	H	1003110	2020080219	139.69	2138.381	0.273	4015.509	4015.509	707.453	77.8

문서 저장소 — KG의 Entity가 가진 값(sourceLocator)으로 CSV의 행을 찾는다

Entity EVI:LAW:별표3
sourceLocator = 별표 3

→ sourceLocator 열이 "별표 3"인 행의 text가 규정 원문이다

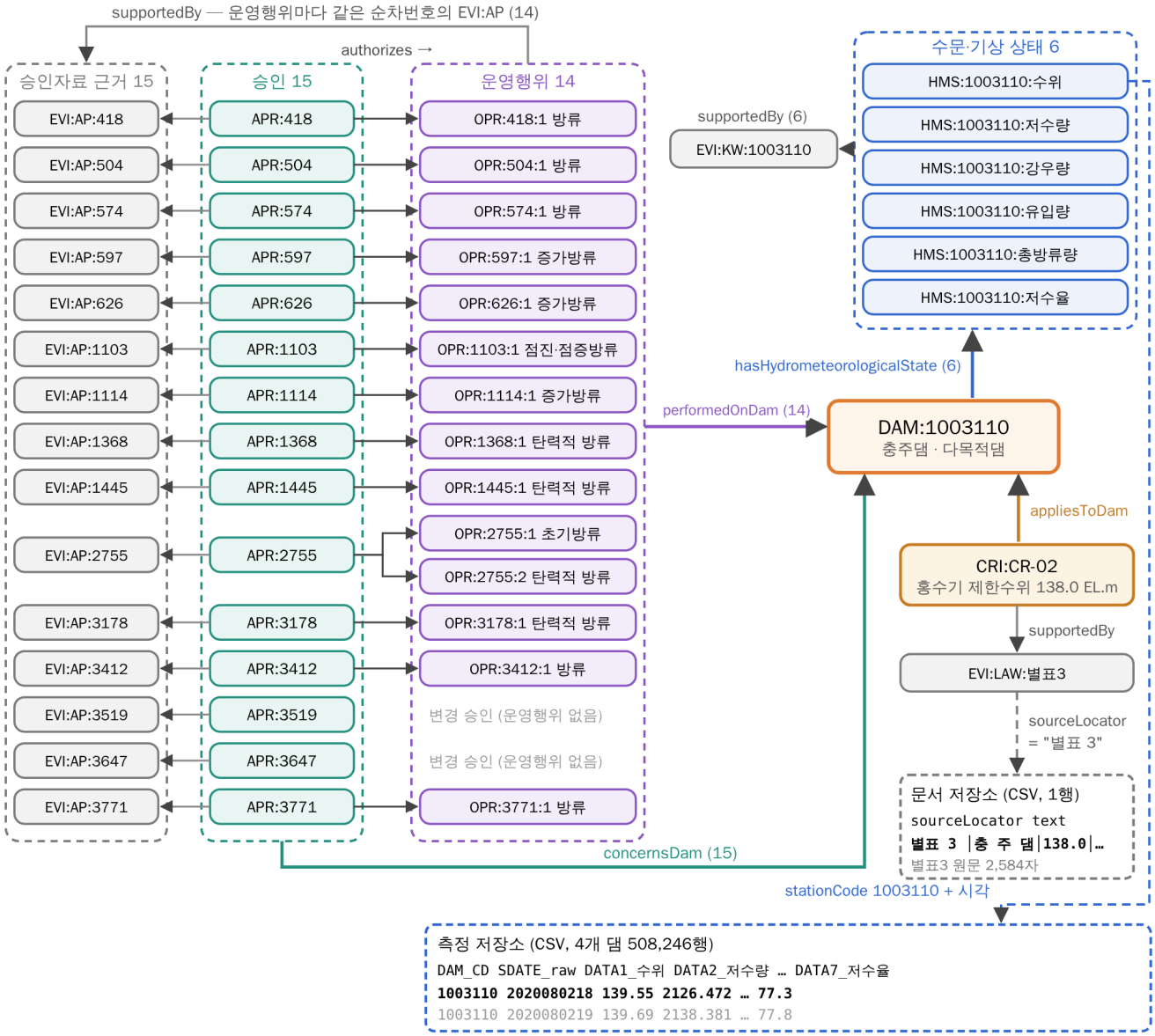
kgv4_documents.csv (1행)

sourceLocator	doc_title	text (2,584자 중 앞부분)
별표 3	댐과 보 등의 연계운영규정 (기후에너지환경부 훈령 제42호, 시행 2026-07-08)	댐의 홍수기 제한수위(제6조) [별표 3] 댐의 홍수기 제한수위(제6조) ┌──┬──┬──┬──┬──┐ |구 분 |댐 종류 |댐 명 |홍수기 제한수위 |시설별 최저 운영수위| | | | | (EL.m) | (EL.m) | ├──┼──┼──┼──┼──┤ |한 강 |다목적댐 |소양강댐 |190.3 |- | |수 계 | |충 주 댐 |138.0 |- | | | |횡 성 댐 |178.2 |- | | ├──┼──┼──┤ | | | |수력발전댐|화 천 댐 |175.0 |- | | | |춘 천 댐 |102.0 |98.0 | | | |의 암 댐 | 70.5 |67.0 | …

그림 10. 외부 저장소 CSV 의 실제 모습과, KG 의 값으로 행을 찾는 방법

3.4 결과: Neo4j 에 저장된 KG

- Neo4j 에 Entity 264 개와 Relation 418 개가 저장된다(표 3).
- 그림 11 은 Neo4j 에 저장된 충주댐의 Entity 53 개와 Relation 을 생략 없이 그린 것이다. 왼쪽 세 점선 상자는 승인자료 근거 15, 승인 15, 운영행위 14 개이고, 한 줄이 승인 한 건이다. 승인마다 왼쪽 근거(supportedBy)와 오른쪽 운영행위(authorizes)로 이어지며, 3519 와 3647 은 변경 승인이라 운영행위가 없다.
- 상자에서 나가는 굵은 선은 상자 안의 Entity 가 모두 가진 Relation 이다. 승인 15 개는 모두 DAM:1003110 에(concernsDam), 운영행위 14 개는 모두 DAM:1003110 에(performedOnDam)와 같은 순차번호의 근거에(supportedBy) 이어진다. 오른쪽 위는 DAM:1003110 의 HMS 6 개와 그 근거, 오른쪽 아래는 기준 CR-02 와 그 근거이다.
- 맨 아래 두 점선 상자는 KG 에 넣지 않은 외부 저장소이고, 점선 화살표는 Relation 이 아니라 KG 의 값으로 그 행을 찾아오는 길이다. HMS 의 stationCode 와 시각으로 측정값을, EVI:LAW:별표 3 의 sourceLocator 로 규정 원문을 가져온다. 4 장의 검색이 이 KG 의 Relation 을 따라가고, 이 점선으로 값을 가져온다.



실선은 Neo4j에 저장된 Relation이고, 점선 상자(승인, 운영행위, 근거, HMS) 안의 Entity는 각각 그 Relation을 가진다(괄호 안은 개수). 맨 아래의 점선 화살표는 Relation이 아니라, KG의 값으로 외부 저장소의 행을 찾아오는 길이다.

그림 11. Neo4j 에 저장된 KG(충주댐 Entity 53 개)와 외부 저장소로 가는 길

표 3. Neo4j 에 저장된 Entity 와 Relation 수

Class	Entity 수	Relation	Relation 수
EvidenceSource	85	supportedBy	175
Approval	80	concernsDam	80
Operation	68	authorizes	68
HydrometeorologicalState	24	performedOnDam	68
Dam	4	hasHydrometeorologicalState	24
Criterion	3	appliesToDam	3
합계	264	합계	418

4. 검색·답변

4.1 따른 원리와 진행 방식

- 따른 원리: Purohit 외(2024)의 GraphAide 는 질의 단계에서 LLM 이 자연어 질문을 부분 그래프 조회 템플릿으로 바꾸고, 이를 Cypher 로 Neo4j 에서 실행한 결과를 LLM 에 맥락으로 주어 답변을 만든다. KG 의 관계를 따라 찾은 근거를 주므로 LLM 이 없는 사실을 지어내는 것(환각)을 줄인다(부록 D.1).
- 우리는 조회 템플릿을 CQ 마다 미리 정해 두었다. LLM 은 질문에서 의도(CQ1~CQ6)와 값(댐, 기간)만 뽑고, 그 다음은 코드가 정해진 순서대로 KG 와 외부 저장소를 조회한다. GraphAide 의 Cypher 자동 생성, 질의 확장, 벡터 검색, 에이전트는 쓰지 않았다.
- 진행 방식: 질문 하나가 의도·값 추출(LLM) → 입력 확인(코드) → KG 검색과 외부 저장소 조회(코드) → 근거 묶음(코드) → 답변(LLM)의 다섯 과정을 거친다. LLM 은 처음과 마지막 두 곳에서 쓰고(같은 모델, 지시문만 다름), 근거를 찾는 가운데 세 과정은 코드가 한다.
- 아래에서는 서버에서 실행한 질문 하나(Qwen3-32B, 2026-10-16)를 따라간다.

4.2 질문에서 근거 묶음까지

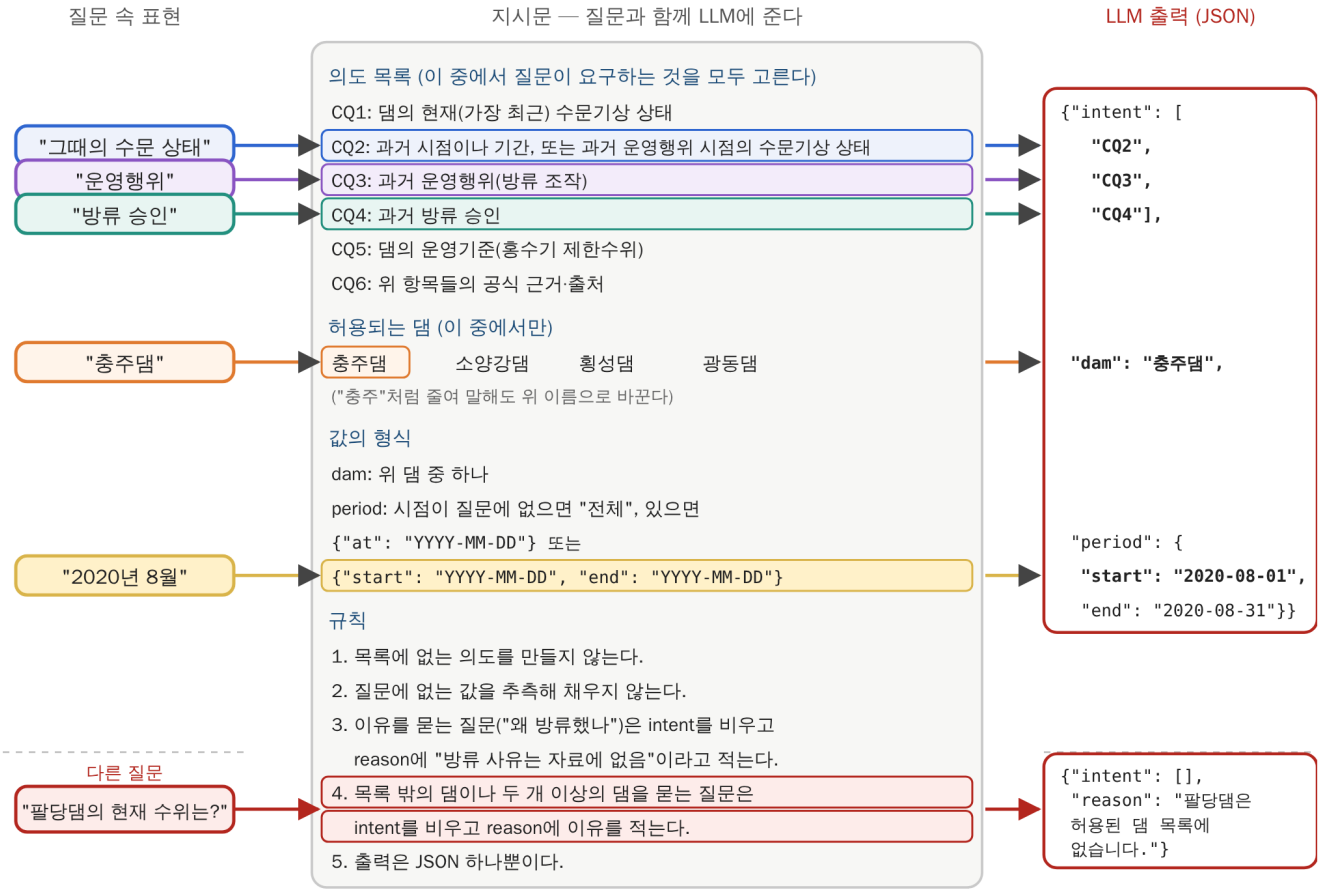
입력

- 질문: "2020 년 8 월 충주댐 방류 승인과 운영행위, 그때의 수문 상태를 알려줘."

과정 1. 의도·값 추출 (LLM)

- 원리: LLM 은 질문을 자유롭게 해석하지 않고, 지시문에 정해 둔 의도 목록 안에서 고른다. 그래서 LLM 에 질문과 함께 지시문을 준다.
- 그림 12 의 가운데가 그 지시문이다. 의도 목록은 온톨로지의 CQ1~CQ6 이며, 질문이 요구하는 것을 모두 고르게 한다. 허용되는 댐은 KG 에 있는 4 개 댐이다. 값의 형식은 다음 과정에서 Cypher 템플릿에 그대로 넣을 수 있는 모양으로 정해 두었다. 규칙 5 개는 목록 밖의 의도나 질문에 없는 값을 만들지 않게 한다.
- 왼쪽은 질문 속 표현, 오른쪽은 LLM 이 낸 JSON 이다. 화살표를 따라가면 "그때의 수문 상태"는 CQ2, "운영행위"는 CQ3, "방류 승인"은 CQ4 에 맞아 intent 가 된다. "충주댐"은 허용되는 댐에 있어 dam 이 되고, "2020 년 8 월"은 기간 형식에 맞춰 2020-08-01 ~ 2020-08-31 이 된다. CQ1, CQ5, CQ6 에 맞는 표현은 없어 고르지 않았다.
- 아래의 다른 질문 "팔당댐의 현재 수위는?"에서는 팔당댐이 허용되는 댐에 없으므로, LLM 이 규칙 4 에 따라 intent 를 비우고 이유를 적었다(같은 서버 실행). 이때는 KG 를 조회하지 않는다.

질문: "2020년 8월 충주댐 방류 승인과 운영행위, 그때의 수문 상태를 알려줘."



LLM 출력 원문: `{"intent": ["CQ2", "CQ3", "CQ4"], "dam": "충주댐", "period": {"start": "2020-08-01", "end": "2020-08-31"}}`

그림 12. 의도·값 추출 — 질문 속 표현을 지시문의 의도 목록, 허용되는 댐, 값의 형식에 맞춰 JSON 으로 적는다

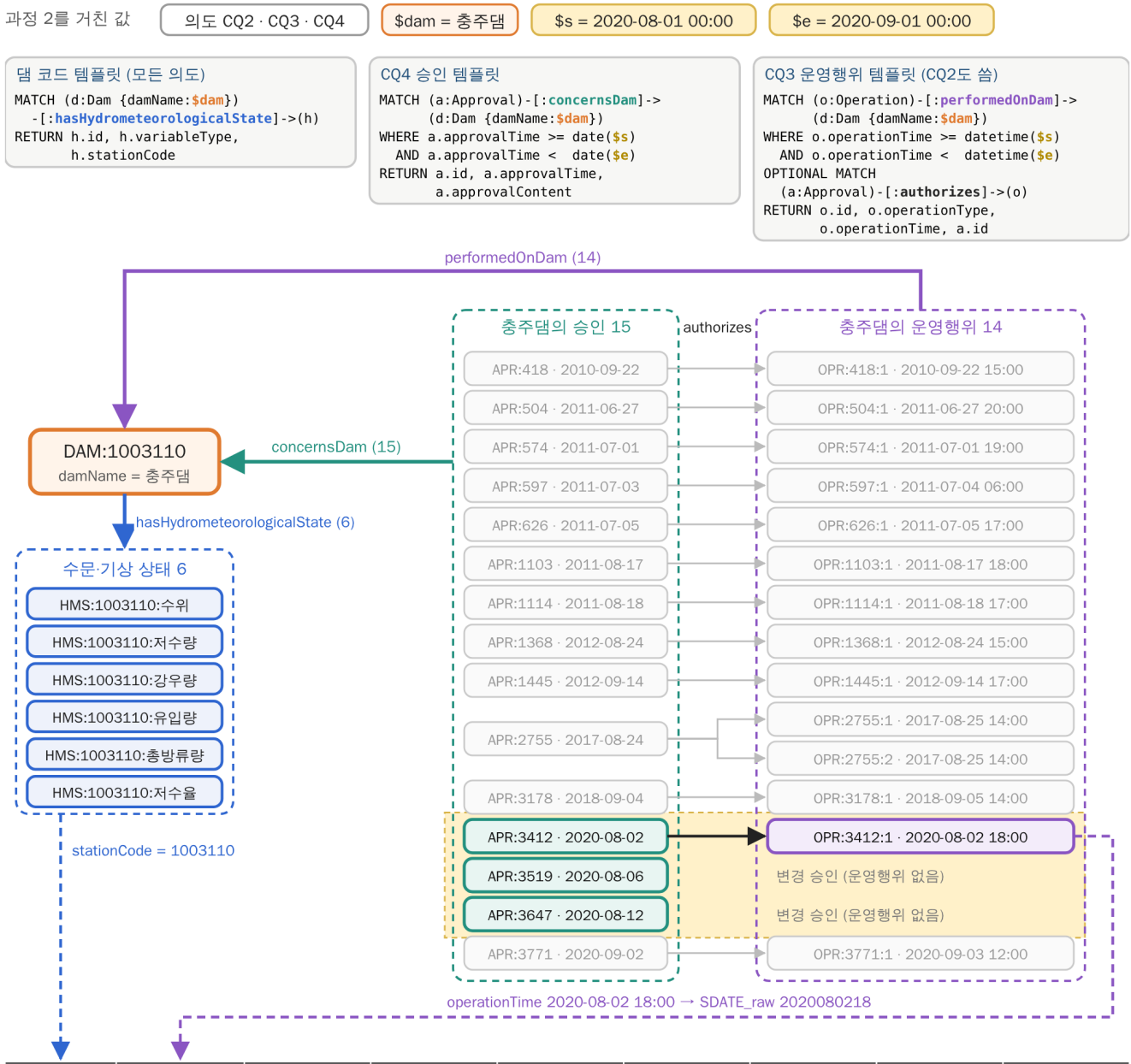
과정 2. 입력 확인 (코드)

- 의도가 CQ1~CQ6 안인지, 댐이 4 개 댐 안인지 확인하고 이름을 표준 댐명으로 바꾼다("충주" → 충주댐).
- 기간의 끝 날짜는 그 날 전체로 읽어 [2020-08-01 00:00, 2020-09-01 00:00)으로 바꾼다. 이 두 시각이 다음 과정의 $s, $e 이다. 확인을 통과하지 못하면 조회하지 않고 "질의 불가: 이유"를 답변 과정으로 넘긴다.

과정 3. KG 검색과 외부 저장소 조회 (코드)

- 원리: 고른 의도마다 미리 정한 Cypher 템플릿에 값($dam, $s, $e)만 넣어 실행한다. 템플릿은 2 장에서 채운 Relation 을 따라가고, 측정값과 원문은 KG 의 Data Property 값으로 외부 저장소(3.3)에서 가져온다.
- 그림 13 의 맨 위는 과정 2 를 거친 값과 이 질문에 쓰는 템플릿 세 개이다. 템플릿의 색 글자는 아래 KG 에서 같은 색의 Relation 이고, $dam, $s, $e 자리에 위의 값이 들어간다.
- 댐 코드 템플릿은 DAM:1003110 에서 hasHydrometeorologicalState 를 따라 HMS 6 개를 찾고 stationCode 1003110 을 얻는다. 측정 저장소를 찾아갈 키이다.
- CQ4 승인 템플릿은 concernsDam 으로 충주댐에 이어진 승인 15 개 가운데 승인일이 $s 이상 $e 미만인 것을 고른다 → APR:3412, APR:3519, APR:3647. 뒤의 둘은 변경 승인이라 운영행위가 없다.
- CQ3 운영행위 템플릿은 performedOnDam 으로 충주댐에 이어진 운영행위 14 개 가운데 시각이 기간 안인 것을 고르고, authorizes 를 거꾸로 따라 그것을 허가한 승인을 찾는다 → OPR:3412:1 ← APR:3412. 노란 띠 밖의 흐린 Entity 는 기간 조건에서 걸러진 것이다.

- CQ2 는 CQ3 과 함께 있으므로 운영행위의 시각마다 그 시각의 상태를 읽는다. 맨 아래 점선처럼 HMS 의 stationCode(1003110)를 DAM_CD 열에서, OPR:3412:1 의 operationTime(2020-08-02 18:00)을 SDATE_raw 열(2020080218)에서 찾으면, 그 행이 그 시각의 측정값이다. 정각이 아닌 시각은 앞뒤 정시 두 행을 함께 읽는다.
- 다른 의도가 있는 질문은 표 4 의 경로를 따른다.



DAM_CD	SDATE_raw	DATA1_수위 (EL.m)	DATA2_저수량 (MCM)	DATA3_강우량 (mm)	DATA4_유입량 (CMS)	DATA5_source_label_unknown	DATA6_총방류량 (CMS)	DATA7_저수율 (%)
1003110	2020080217	139.37	2111.215	0.711	4693.654	4693.654	707.265	76.8
1003110	**2020080218**	**139.55**	**2126.472**	**1.901**	**4945.201**	**4945.201**	**707.145**	**77.3**
1003110	2020080219	139.69	2138.381	0.273	4015.509	4015.509	707.453	77.8

측정 저장소 kwater_mywater_hydr_H_aligned_wide.csv.gz의 행 (앞의 source_file 등 3열은 생략). 고른 행의 값이 [MEA:1003110:2020080218]이 된다.

실선은 Neo4j에 저장된 Relation, 노란 띠는 $s~$e 기간 안의 Entity이다. 진하게 칠한 Entity가 이 질문에서 찾은 것이고 흐린 것은 기간 밖이라 걸러진 것이다. 점선은 KG의 값으로 측정 저장소의 행을 찾아가는 길이다.

그림 13. KG 검색과 외부 저장소 조회 — 템플릿이 따라가는 Relation 과 기간 조건, KG 의 값으로 찾는 측정 저장소의 행

표 4. 의도별 Cypher 템플릿이 따라가는 Relation 과 외부 저장소

의도	따라가는 Relation (Cypher 템플릿)	외부 저장소
CQ1 현재 상태	Dam → hasHydrometeorologicalState → HMS (stationCode)	측정: 저장된 가장 최근 시각의 행
CQ2 과거 상태	(CQ3 과 함께면) 운영행위의 operationTime	측정: 그 시각의 행, CQ3 이 없으면 기간 안의 행
CQ3 운영행위	Operation → performedOnDam → Dam, Approval → authorizes → Operation	—
CQ4 승인	Approval → concernsDam → Dam	—
CQ5 운영기준	Criterion → appliesToDam → Dam, Criterion → supportedBy → EvidenceSource	문서: sourceLocator 의 원문
CQ6 근거	조회한 Entity → supportedBy → EvidenceSource	—

과정 4. 근거 묶음 (코드)

- 조회 결과를 의도별 구역으로 나누고, 항목마다 출처 번호를 붙인다. 출처 번호는 KG 의 Entity ID 이고, 측정값은 [MEA:댐 코드:시각]이다.
- 값은 저장소의 원값 그대로 쓰고 계산한 값은 넣지 않는다. 끝에 근거 사용 원칙 3 문장을 붙인다.

```
[질문] 2020년 8월 충주댐 방류 승인과 운영행위, 그때의 수문 상태를 알려줘.
[대상] 충주댐, 기간 2020-08-01 ~ 2020-08-31

[근거 1. 운영행위와 허가한 승인]  (CQ3) 운영행위의 시각은 비고에 적힌 시각, 없으면 승인된 방류 시작 시각이다.
[OPR:3412:1] 방류, 시각 2020-08-02 18:00 ← 승인 [APR:3412]

[근거 2. 운영 시각의 상태]  (CQ2)
[MEA:1003110:2020080218] 2020-08-02 18:00  수위 139.55 EL.m, 저수량 2126.472 MCM, 강우량 1.901 mm,
   유입량 4945.201 CMS, 총방류량 707.145 CMS, 저수율 77.3 %

[근거 3. 방류 승인]  (CQ4)
[APR:3412] 승인일 2020-08-02: 방류시작 2020-08-02 18:00; 접수방류량 3000㎥/s
[APR:3519] 승인일 2020-08-06: 방류시작 2020-08-02 18:00; 접수방류량 7000㎥/s; 비고: 방류기간 (당초) … (변경)
…
[APR:3647] 승인일 2020-08-12: 방류시작 2020-08-06 06:00; 접수방류량 7000㎥/s; 비고: 수문방류 변경승인(방류기
간연장) …

[근거 사용 원칙] 근거에 있는 내용만 사실로 쓰고 출처 번호를 붙인다 / 비교·해석은 판단임을 밝힌다 / 없으면 "자료에 없
음"
```

의도별 구역 출처 번호 = KG의 Entity ID, 측정값은 MEA 원칙 3문장

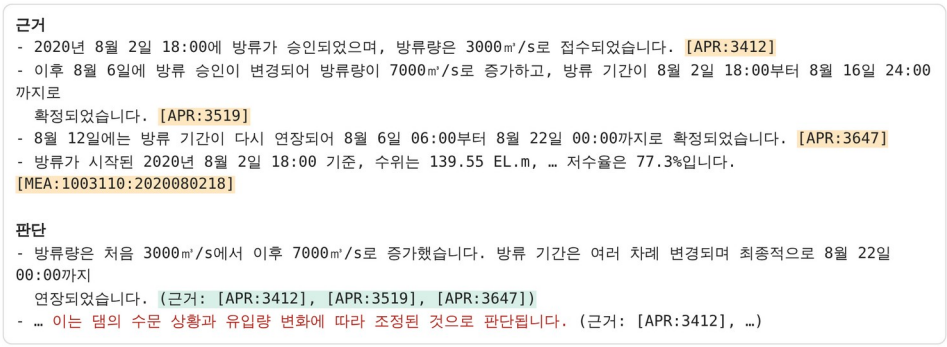
그림 14. 근거 묶음 — LLM 에 주는 근거 전체

과정 5. 답변 (LLM)

- LLM 에 답변 지시문과 근거 묶음을 준다. 지시문: 근거 묶음에 있는 내용만 사실로 쓰고 출처 번호를 붙인다, 비교·해석은 "판단"으로 나누고 쓴 근거를 적는다, 없으면 "자료에 없음".
- 주의 사항도 함께 준다: 운영행위 시각은 비고의 시각이거나 승인된 방류 시작 시각이다, "현재"는 저장된 가장 최근 시각이며 실시간이 아니다, 방류 사유는 자료에 없으므로 단정하지 않는다.

4.3 결과: 답변

- 그림 15 가 이 질문의 답변이다. "근거"에는 승인 세 건의 방류량과 기간이 [APR:3412], [APR:3519], [APR:3647] 과 함께, 방류 시작 시각의 수문 상태가 [MEA:1003110:2020080218]과 함께 쓰여 있다.
- "판단"에는 방류량이 3000㎥/s 에서 7000㎥/s 로 늘고 기간이 연장되었다는 비교가 쓴 근거와 함께 있다. 빨간 글씨 문장은 자료에 없는 사유를 추정한 것으로, 지시문의 "사유를 단정하지 않는다"를 어겼다.
- 근거 묶음의 운영행위 [OPR:3412:1]은 답변에 출처 번호로 쓰이지 않았다.



사실마다 출처 번호 판단마다 쓴 근거 빨간 글씨: 자료에 없는 사유를 추정한 문장 (지시문의 "사유를 단정하지 않는다"를 어김)

그림 15. 답변 — 근거(출처 번호)와 판단(쓴 근거)으로 나뉜다

5. 검토할 점과 다음 계획

1. 답할 수 있는 질문의 범위 (온톨로지, 검색·답변)

- 온톨로지는 CQ1~CQ6 에 답할 수 있는 구조로 만들었고, 검색은 CQ 마다 정한 Cypher 템플릿으로 한다(4 장). 그래서 지금 구조는 질문의 의도가 CQ1~CQ6 에 해당할 때만 답할 수 있다.
- 의도 목록에 없는 질문은 과정 1 에서 intent 가 비어 조회하지 않는다. 예: 방류 사유를 묻는 질문, 4 개 댐 밖의 댐이나 두 댐을 함께 묻는 질문.

2. 과제 발표와 시연

- 11/20 과제 발표와 11/26 시연은 지금 구조(4 장)로 한다.
- 서버 실행 결과는 질문 몇 개를 새로 만들어 다시 답변을 받은 뒤 정리한다.

3. 연구

- 온톨로지 생성 방식을 다시 검토한다.
- 검색·답변과 검증에 대한 선행 논문을 더 찾아 읽고 지금 방법과 비교한 뒤, 더 좋은 결과를 내는 방법을 적용한다. 검증은 11/02 회의에 따라 다른 연구들이 KG·온톨로지를 어떻게 검증했는지부터 조사한다.

부록 A. 용어 정리

표 5. 용어 정의

용어	본 연구에서의 정의
Relation 채우기	온톨로지가 허락한 Class 사이의 관계 가운데, 자료 속 키로 Entity 사이의 실제 관계(시작 Entity → 끝 Entity)를 골라 잇는 일.
개체 해소	자료마다 다르게 적힌 같은 대상(충주, 충 주 댐, 1003110)을 하나의 기준 식별자에 대응시키는 일. 문헌의 entity disambiguation.
기준 식별자	같은 대상을 가리키는 표기들을 모으는 기준 ID. 본 연구는 K-water 댐 코드, GraphAide 는 Wikidata 의 QID.
Neo4j	Entity 와 Relation 을 그래프(점과 선) 모양으로 저장하는 데이터베이스.
Cypher	Neo4j 에 Entity 와 Relation 을 넣고 찾을 때 쓰는 명령 문장. MERGE(없으면 만듦), MATCH(찾음), SET(값을 넣음) 등.
지시문	LLM 에 질문과 함께 주는 지침 글(프롬프트). 의도·값 추출용과 답변용이 따로 있다.
조회 템플릿	CQ 마다 미리 정해 둔 Cypher 문. 댐 이름과 기간만 바꿔 넣어 실행한다.
의도	질문이 요구하는 CQ(CQ1~CQ6). 한 질문에 여러 개가 있을 수 있다.
근거 묶음	KG 와 저장소에서 찾은 근거를 의도별 구역과 출처 번호를 붙여 LLM 에 주는 글.
출처 번호	근거 항목마다 붙이는 표시. KG 의 Entity ID 이거나 측정값의 [MEA:댐 코드:시각].

용어	본 연구에서의 정의
환각	LLM 이 자료에 없는 사실을 지어내는 것.

부록 B. Ontology 가 답해야 할 질문과 관계

표 6. 세부 질문 (CQ1~CQ6)

번호	질문 대상	세부 질문
CQ1	현재 수문·기상 상태	대상 댐의 현재 또는 기준시점의 강우, 수위, 유입량, 방류량 등 수문·기상 상태는 무엇인가?
CQ2	과거 수문·기상 상태	선택된 과거 시점 또는 사례에서 대상 댐의 강우, 수위, 유입량, 방류량 등 수문·기상 상태는 어떠했는가?
CQ3	과거 운영행위	해당 과거 시점 또는 사례에서 실제로 수행된 댐 운영행위는 무엇이었는가?
CQ4	과거 방류 승인	해당 과거 시점 또는 사례에서 어떤 방류 승인이 있었으며, 그 승인내용은 무엇인가?
CQ5	현재 운영기준	현재 대상 댐에 적용되는 운영기준은 무엇이며, 현재 운영 판단과 관련된 기준값은 무엇인가?
CQ6	공식 근거	각 수문·기상 상태, 운영행위, 승인내용 및 운영기준을 확인할 수 있는 공식 자료 또는 근거문서는 무엇인가?

표 7. Relation (9 개)

Subject	Relation	Object	의미
Dam	hasHydrometeorologicalState	HydrometeorologicalState	댐이 해당 수문·기상 변수 정보와 연결됨
Operation	performedOnDam	Dam	운영행위가 특정 Dam 에서 수행됨
Approval	authorizes	Operation	Approval 이 특정 Operation 을 승인함
Approval	concernsDam	Dam	Approval 이 특정 Dam 에 관한 것임
Criterion	appliesToDam	Dam	운영기준이 특정 Dam 에 적용됨
HydrometeorologicalState	supportedBy	EvidenceSource	상태를 확인할 수 있는 근거자료
Operation	supportedBy	EvidenceSource	운영행위를 확인할 수 있는 근거자료
Approval	supportedBy	EvidenceSource	승인을 확인할 수 있는 근거자료
Criterion	supportedBy	EvidenceSource	운영기준을 확인할 수 있는 근거자료

부록 C. 10/30 자료까지의 결과: 자료 준비, Class·Data Property 채우기

- 자료 준비: 승인 기록 3,929 행에서 중복 7 행을 지우고 대상 4 개 댐의 80 행을 남겼다. K-water 시간자료 두 파일을 이어 붙여 4 개 댐의 508,246 행을 측정 저장소에, 규정 XML 에서 꺼낸 별표 3 원문을 문서 저장소에 두었다.
- Class·Data Property 채우기: 정형 표는 구조화 자료 매핑으로, 승인 비고와 별표 3 은 비정형 텍스트 추출로 Entity 와 속성값을 만들었다(표 8). 결과는 kgv4_entities.csv(Entity 264 개)와 kgv4_properties.csv(속성값 616 개)이다.
- 이 단계에서 이어진 Relation 은 같은 행·같은 자료에서 함께 만든 근거(supportedBy) 175 개뿐이었다. 나머지 Relation 243 개를 2 장에서 채웠다.

표 8. 자료별로 채운 Class, Entity 수, Data Property

방법	자료	Class (Entity 수)	Data Property (속성값 수)
구조화 자료 매핑	승인 기록 (80 행)	Approval (80)	approvalTime, approvalContent (160)
		EvidenceSource 승인자료 (80)	sourceType, sourceTitle, sourceLocator (240)
	K-water 일자료 (대상 댐 4)	Dam (4)	damName, damType (8)
	K-water 시간자료 (변수 6)	HydrometeorologicalState (24)	stationCode, variableType (48)
		EvidenceSource 측정자료 (4)	sourceType, sourceTitle, sourceLocator (12)
비정형 텍스트 추출	승인 비고	Operation (68)	operationTime, operationType (136)
	규정 별표 3	Criterion (3)	criterionType, criterionValue, unit (9)
		EvidenceSource 규정 (1)	sourceType, sourceTitle, sourceLocator (3)
합계		Entity 264	속성값 616

부록 D. 근거 문헌의 원리

D.1 Purohit 외(2024) — Relation 채우기(개체 해소), 검색·답변

- 무엇인가: 여러 출처의 정형·비정형 자료로 KG 를 만들고, 그 KG 를 조회·추론하는 질의 시스템 GraphAide 를 제안한 학회 논문이다 (IEEE BigData 2024). OpenAI LLM, Chroma 벡터 DB, Neo4j, LangChain 으로 구현했다.
- KG 생성 단계의 개체 해소: 기준 지식베이스로 DARPA Wikidata(DWD)를 쓰고, QID 마다 설명 문서를 만들어 벡터 DB 에 넣었다. 온톨로지와 DWD 에 기반한 에이전트가 글에서 대상의 표기를 뽑고, RAG 기반 후보 재순위로 가장 가까운 QID 를 고른다. → 기준 식별자에 대응시키는 원리만 따랐고, 벡터 DB 와 LLM 재순위는 쓰지 않았다.
- 질의 단계: LLM 이 질문을 여러 형태로 확장하고, 이를 부분 그래프 조회 템플릿으로 바꿔 Cypher 를 자동 생성한다. 그래프 조회 결과와 벡터 검색 결과를 합쳐 LLM 의 맥락으로 주고, 답변을 모아 하나의 응답을 만든다. 정해진 순서(chain)와 에이전트를 함께 쓴다. → KG 조회 결과를 LLM 의 근거로 주는 원리를 따랐고, Cypher 는 CQ 마다 미리 정한 템플릿으로 대신했다.
- 평가: RAGAS 지표와 전문가 정성 평가를 쓴다고 밝혔으며, 정식 평가는 향후 과제로 두었다.

D.2 Huang 외(2026) — 저장

- 추출한 지식을 JSON 으로 통합하고, DDO 의 의미·계층 관계에 따라 Python 스크립트가 JSON 키-값을 읽어 Cypher 문을 만든다. 이 문들로 DDKG 의 Entity 를 일괄 생성하고 Entity 사이의 관계를 만들어 Neo4j 에 적재한다.
- 저장소로 Neo4j 를 고른 이유: Cypher 로 조회·추가·삭제·수정이 효율적이고, 다중 홉 추론과 복잡한 질의에 강하다.
- 구축한 DDKG 는 Entity 201,362 개, Relation 953,447 개이다. → JSON 대신 CSV 를 모은 형태로 쓴 것 외에는 같은 순서(모음 → 스크립트로 Cypher 생성 → 일괄 적재)를 따랐다.

부록 E. 참고문헌

- Huang, J., Hou, W., Zhang, S., Du, Z., & Yin, J. (2026). Dam defects ontology and knowledge graph construction from multi-source hazard data using large language models. Advanced Engineering Informatics, 76, 104987.
- Purohit, S., Chin, G., Mackey, P. S., & Cottam, J. A. (2024). GraphAide: Advanced Graph-Assisted Query and Reasoning System. 2024 IEEE International Conference on Big Data. https://doi.org/10.1109/BigData62323.2024.10825705